

Séance 3 : Qualité en flux et détection d'anomalies

Owner	 louis reynouard
Tags	

Contexte

Vous avez une couche Silver streaming fonctionnelle. Les données sont typées, enrichies géographiquement, et les fenêtres temporelles sont paramétrées. En streaming, on ne peut pas rejouer les données pour corriger une erreur de qualité a posteriori. Quand une ligne corrompue passe, elle est dans la table. Quand une ligne valide est rejetée, elle est perdue.

Cette séance introduit les Stream-Stream Joins et la détection d'anomalies en flux.

1. Qualité en streaming : contraintes spécifiques

En batch (Spark Core S3), vous avez implémenté une quarantaine : les lignes invalides allaient dans une table séparée. Cette logique reste valide en streaming, avec une contrainte supplémentaire : le routage ne doit pas arrêter le flux.

Si votre vérification de qualité lève une exception non gérée, le micro-batch échoue, et Spark retente indéfiniment. Le pipeline est bloqué.

Approche correcte : ajouter une colonne de flag via des conditions `when` / `otherwise` sur le DataFrame, puis filtrer en deux DataFrames distincts (valide et quarantaine). Aucune exception n'est levée : chaque ligne est simplement routée. Chaque DataFrame écrit vers une table Delta distincte avec son propre checkpoint.

2. Idempotence et Exactly-Once

En streaming, un micro-batch peut être rejoué si le job redémarre après un crash au milieu du traitement. Sans garantie d'exactitude, vous risquez d'écrire deux fois les mêmes lignes.

Les transactions ACID de Delta Lake

Delta Lake est un format de stockage qui ajoute une couche transactionnelle au-dessus de fichiers Parquet. ACID désigne quatre propriétés que tout système transactionnel sérieux doit garantir.

- **Atomicité** : une opération d'écriture réussit en totalité ou ne réussit pas du tout. Si le cluster crash pendant l'écriture d'un micro-batch, aucune ligne partielle n'est visible dans la table. Le prochain lecteur voit l'état avant l'écriture ou après, jamais un état intermédiaire.
- **Cohérence** : une transaction ne peut pas amener la table dans un état invalide. Les contraintes de schéma sont respectées à chaque écriture.
- **Isolation** : si plusieurs jobs lisent et écrivent en même temps (par exemple un job streaming qui écrit et un job batch qui lit la même table), chacun voit un état cohérent de la table. Delta Lake utilise un mécanisme de versioning par fichier journal (`_delta_log`) : chaque transaction est enregistrée comme une nouvelle entrée dans ce journal, et les lecteurs consultent la dernière version validée.
- **Durabilité** : une fois qu'une transaction est validée (commitée dans le `_delta_log`), elle est permanente même en cas de panne immédiatement après.

Checkpoint et sink Delta : les deux verrous de l'Exactly-Once



Rappel:

Le sink, c'est la **destination d'écriture** d'un job streaming — là où Spark envoie les résultats de chaque micro-batch.

En batch (Spark Core), vous écriviez avec `.write`. En streaming, l'équivalent s'appelle le sink et s'exprime avec `.writeStream`. Ce qui suit `.writeStream` définit complètement le sink : le format (`delta` , `csv` , `json` ...), le chemin ou la table de destination, le mode de sortie, et le checkpoint.

Le checkpoint enregistre l'offset de lecture (jusqu'où les fichiers ont été lus) et l'état interne du job (watermarks, agrégations). Au redémarrage, Spark lit le checkpoint, sait que le batch N a été traité, et repart du batch N+1. Mais cela suppose que le batch N a bien été écrit dans le sink.

Écrire dans un CSV ou un dossier Parquet non-Delta, c'est créer des fichiers un par un, sans garantie atomique. Si le cluster crash après le premier fichier et avant le second, des données partielles restent sur le disque. Spark rejoue le batch au redémarrage, ce qui duplique le premier fichier.

Avec Delta, l'écriture est atomique : soit tous les fichiers Parquet sont écrits et l'entrée est validée dans le `_delta_log`, soit rien n'est validé. En cas de crash avant le commit, Delta ignore les fichiers partiels. Spark rejoue le batch sans duplication.

Utilisez `.toTable()` avec format Delta ou `.save()` avec `format("delta")`. Un sink CSV, JSON ou Parquet non-Delta ne garantit pas l'Exactly-Once.

3. Stream-Stream Joins

Jusqu'ici, vous avez fait des Stream-Static Joins : un flux + une table statique. Parfois, les deux sources sont des flux. C'est le Stream-Stream Join, et il est significativement plus complexe.

Le problème fondamental

Pour joindre deux flux sur une clé commune, Spark doit garder en mémoire les événements des deux flux en attente d'une correspondance. Sans watermark, cet

état grossit indéfiniment. Avec watermark sur les deux flux, Spark décide d'un horizon au-delà duquel il abandonne la recherche et libère la mémoire.

La jointure s'exprime comme une jointure classique, mais les deux DataFrames sont des flux. Chacun porte son propre `.withWatermark()`. La condition de jointure inclut généralement une contrainte temporelle (les deux événements doivent être proches dans le temps) pour éviter que l'état ne grossisse trop.

Exemple illustratif (domaine e-commerce)

```
# Flux A : commandes passées
df_orders = spark.readStream.format("delta") \
    .table("orders") \
    .withWatermark("order_time", "10 minutes")

# Flux B : paiements reçus
df_payments = spark.readStream.format("delta") \
    .table("payments") \
    .withWatermark("payment_time", "10 minutes")

# Rapprocher chaque commande à son paiement dans une fenêtre
# de 5 minutes
df_matched = df_orders.join(
    df_payments,
    (df_orders.order_id == df_payments.order_id) &
    (df_orders.order_time.between(
        df_payments.payment_time - expr("INTERVAL 5 MINUTE
S"),
        df_payments.payment_time
    )),
    "left"
)
```

Le principe est identique quelle que soit la clé de jointure. Ce qui change selon votre cas : la clé commune, la tolérance temporelle, et le type de jointure (`inner`, `left`).

Limitations sur compte gratuit

Les Stream-Stream Joins consomment beaucoup de mémoire : Spark maintient l'état des deux flux jusqu'à l'horizon du watermark. Sur un cluster single-node avec 15-30 Go de RAM, limitez la simulation à 20 batches maximum et nettoyez les checkpoints entre les tests.

4. Détection d'anomalies en flux

Plutôt que de simplement filtrer les anomalies évidentes (montant négatif), on peut détecter des anomalies statistiques en comparant chaque trajet à une référence batch.

La référence batch, c'est votre table Gold de Spark Core : vous avez calculé la moyenne et l'écart-type du montant par borough. Un trajet dont le montant dépasse la moyenne + $3 \times$ l'écart-type est statistiquement suspect. Ce calcul s'implémente via un Stream-Static Join entre le flux Silver et votre table de statistiques batch, suivi d'une colonne de z-score calculée à la volée.

Ce pattern relie directement Spark Core (les stats batch) au streaming (la détection temps réel). C'est le modèle de monitoring utilisé en production dans les équipes MLOps.

Attention au cas où le z-score est nul : si un borough est absent de vos stats batch, la division produit `NaN`. Gérez ce cas explicitement dans votre flag.

TP Séance 3 : Qualité temps réel et détection d'anomalies

Ce qui vous est donné

Le simulateur configuré en mode `chaos` avec `chaos_ratio: 0.20`. Ce sont les seuls paramètres imposés. Le reste de la configuration est votre décision.

Ce que vous devez construire

Un pipeline de qualité avec quarantaine. À partir de `silver_stream_taxi`, produire deux tables distinctes : `gold_stream_taxi` pour les lignes valides, `quarantine_stream_taxi` pour les lignes rejetées. Le routage se fait via une colonne `quality_flag` que vous définissez. Les critères de rejet sont votre décision : a minima les anomalies évidentes (montant négatif, distance nulle, LocationID manquant), et au moins un critère statistique basé sur vos stats Spark Core. Deux jobs, deux checkpoints distincts.

Un dashboard de qualité. Pendant la simulation, relevez le taux de rejet global et la distribution des types d'anomalies à deux moments différents. Les deux sorties doivent figurer dans le notebook avec un commentaire sur l'évolution.

Un test de résilience. Arrêtez manuellement le job Gold, relancez-le, et prouvez par le compte de lignes que les données ne sont pas dupliquées. Expliquez dans une cellule Markdown le mécanisme qui garantit cela.

Contraintes techniques

Le pipeline ne doit jamais crasher sur une donnée corrompue. Si une exception non gérée est levée, le micro-batch entier échoue et Spark retente indéfiniment. Le test de résilience est éliminatoire : un pipeline qui ne survit pas à un restart ne sera pas noté.

Livrable : Notebook `S3_Qualite_Stream_TRIGRAMME`

Les deux jobs avec leurs justifications. Le dashboard de qualité à deux moments. Le test de résilience avec le compte avant et après. Deux cellules Markdown répondant aux questions suivantes.

Question 1. Vous observez que le taux de rejet dû à `ANOMALIE_STATISTIQUE` varie fortement d'un micro-batch à l'autre. Proposez une explication. Quel ajustement permettrait de stabiliser ce taux ?

Question 2. Ignorer définitivement les données en quarantaine ou les réinjecter après correction : quels sont les arguments pour chaque option dans le contexte taxi ? Quelle serait votre décision et pourquoi ?

Grille d'évaluation

Critère	Insuffisant	Attendu	Excellent
Pipeline sans crash	Job s'arrête sur donnée corrompue	Gestion via <code>quality_flag</code>	gestion des nulls dans le z-score
Quarantaine	Absente	Table peuplée	distribution par type
Test résilience	Non réalisé	Restart sans duplication prouvé	explication du mécanisme (checkpoint + Delta)
Q1 variabilité	Pas de réponse	Explication qualitative	Explication + proposition implémentable
Q2 quarantaine	Pas de réponse	Choix justifié	analyse des implications opérationnelles

Le test de résilience est éliminatoire : un pipeline qui ne survit pas à un restart ne sera pas noté.